

总体刚度组装与桁架结构求解

第五次作业

指导教师：肖姚，郭然

姓名：李泽勇 202311012114

班级：工力 231

学院：建筑工程学院

目录

1.总体刚度方程的建立过程	1
1.1 编号	1
1.2 在局部坐标系下，建立每个单元 e 的刚度方程:	1
1.3 组装总体刚度矩阵	1
1.4 引入边界条件与方程求解	1
2.对号矩阵 LM 的生成方法	1
2.1 对号矩阵的作用	1
2.2 LM 的生成方法	1
3.直接组装算法说明	3
3.1 算法目标	3
3.2 代码	3
3.3 代码解释	4
4.位移边界条件的处理方法	5
5.两个验证算例的输入数据，输出结果和结果校核	8
5.1 输入数据	8
5.2 输出结果	10
5.3 结果校核	12
6.总体刚度矩阵性质分析	12
6.1 算例 1	12
6.2 算例 2	13
7.总体刚度矩阵第 j 列物理意义说明	13

1. 总体刚度方程的建立过程

1.1 编号

将连续结构划分为有限个单元，并为所有节点进行整体编号

1.2 在局部坐标系下，建立每个单元 e 的刚度方程：

$$k^e \delta^e = f^e \quad (\text{式 1.1})$$

k^e : 单元刚度矩阵，通过材料本构和几何参数计算得出。

δ^e : 单元节点位移。

f^e : 单元节点力。

1.3 组装总体刚度矩阵

将所有单元的刚度，根据其连接关系，准确地累加到全局矩阵的正确位置上。

1.4 引入边界条件与方程求解

通过施加适当的位移边界条件将奇异的总体刚度矩阵变为非奇异矩阵，之后可通过缩减法，修改法，罚函数法进行求解。

2. 对号矩阵 LM 的生成方法

2.1 对号矩阵的作用

定义了单元局部自由度编号与结构总体自由度编号之间的映射关系。

2.2 LM 的生成方法

2.2.1 初始化 LM 矩阵

```
n=self.nen*self.ndof
```

```
self.LM=np.zeros((self.nel,n),dtype=int)
```

n : 计算每个单元的总局部自由度数=每个单元的节点数 (nen) $\times$ 每个节点的自由度数 ($ndof$)

对于一维杆单元: $nen=2$ (2 个节点), $ndof=1$ (每个节点 1 个轴向自由度), $n=2$

对于二维桁架单元: $nen=2$ (2 个节点), $ndof=2$ (每个节点 2 个平动自由度), $n=4$

$self.LM$: 创建一个形状为 (nel,n) 的全零整数矩阵

行数=单元总数 (nel), 每行对应一个单元

列数=每个单元的自由度数 (n), 每列对应单元的一个局部自由度

2.2.2 三重循环填充 LM 矩阵

```
for e in range(self.nel):           #遍历所有单元
    for i in range(self.nen):       #遍历单元的所有节点
        node = self.IEN[e, i]      #获取当前节点在总体节点中的编号
        for j in range(self.ndof):  #遍历节点的所有自由度
            local_idx = i * self.ndof + j
            global_idx = node * self.ndof + j
            self.LM[e, local_idx] = global_idx
```

for e in range(self.nel):

逐个处理每个单元，e 是当前处理的单元编号（从 0 开始）

for i in range(self.nen):

遍历当前单元的所有节点，i 是节点在单元局部的编号

从连接数组 self.IEN 中获取该局部节点对应的总体节点编号：node=self.IEN[e,i]

for j in range(self.ndof):

遍历当前节点的所有自由度方向，j 是自由度方向编号

对于桁架单元，j=0 表示 X 方向，j=1 表示 Y 方向

对于杆单元，j 始终为 0（只有轴向自由度）

2.2.3 索引计算公式

```
local_idx = i * self.ndof + j
global_idx = node * self.ndof + j
```

local_idx（局部自由度编号）：

将局部节点编号 i 和自由度方向 j 组合成单元内的统一编号

例如二维桁架（ndof=2）：

单元第一个节点（i=0）：j=0→local_idx=0(UX₁)

单元第一个节点（i=0）：j=1→local_idx=1(UY₁)

单元第二个节点（i=1）：j=0→local_idx=2(UX₂)

单元第二个节点（i=1）：j=1→local_idx=3(UY₂)

global_idx（总体自由度编号）：

将总体节点编号 **node** 和自由度方向 **j** 组合成结构总体的统一编号

使用公式：节点编号×每节点自由度数+自由度方向

例如：节点 5（编号从 0 开始为 4）的 Y 方向自由度（ $j=1$ ），总体编号为： $4 \times 2 + 1 = 9$

2.2.4 存储映射关系

`self.LM[e, local_idx] = global_idx`：

将映射关系存储到 LM 矩阵中

含义：单元 **e** 的第 **local_idx** 个局部自由度，对应整体结构的第 **global_idx** 个总体自由度

3.直接组装算法说明

3.1 算法目标

将结构中所有离散单元的刚度矩阵（**ke**），根据其几何连接关系，系统地叠加组合成一个统一的、代表整体结构力学特性的总体刚度矩阵（**K**）。

3.2 代码

```
def assemble_global_stiffness(self):
    """组装总体刚度矩阵"""
    n = self.nnp * self.ndof
    self.K = np.zeros((n, n))
    print("\n 开始组装总体刚度矩阵...")
    for e in range(self.nel):
        ke, L, c, s = self.compute_element_stiffness(e)
        # 直接组装
        for a in range(ke.shape[0]):
            i = self.LM[e, a]
            for b in range(ke.shape[1]):
                j = self.LM[e, b]
                self.K[i, j] += ke[a, b]
```

3.3 代码解释

3.3.1 初始化总体刚度矩阵

```
n = self.nnp * self.ndof
self.K = np.zeros((n, n))
```

计算总体自由度数 n :

`self.nnp` 是节点总数。

`self.ndof` 是每个节点的自由度数（例如，二维桁架中 `ndof=2`，代表每个节点的 X 和 Y 方向位移）。

`self.K = np.zeros((n, n))`:

创建零矩阵 `self.K`:

使用 `np.zeros` 创建一个大小为 (n,n) 的全零矩阵，作为最终总体刚度矩阵的“容器”。

3.3.2 遍历所有单元

```
for e in range(self.nel):
```

变量 e 从 0 递增到 `self.nel-1`，目的是依次处理结构中的每一个单元，将其刚度贡献累加到总刚 `self.K` 中。

3.3.3 获取当前单元的刚度矩阵

```
ke, L, c, s = self.compute_element_stiffness(e)
```

调用 `compute_element_stiffness(e)` 函数，为当前单元 e 计算其在整体坐标系下的单元刚度矩阵 `ke`，并同时返回单元长度 L 和方向余弦 (c,s) 。

3.3.4 组装循环

```
for a in range(ke.shape[0]):
    i = self.LM[e, a]
    for b in range(ke.shape[1]):
        j = self.LM[e, b]
        self.K[i, j] += ke[a, b]
```

双重循环遍历 `ke` 的所有元素:

外层循环索引 a 遍历单元刚度矩阵 `ke` 的所有行。

内层循环索引 b 遍历单元刚度矩阵 `ke` 的所有列。

对于 $ke[a,b]$ 代表该单元第 a 个局部自由度与第 b 个局部自由度之间的刚度系数，我们需要确定它应该被累加到总体矩阵 $self.K$ 的哪个位置。

查询映射表（对号矩阵 LM ）：

$i = self.LM[e,a]$ ：查询对号矩阵。 $self.LM[e,a]$ 存储了单元 e 的第 a 个局部自由度所对应的总体自由度编号。 i 就是 $ke[a,b]$ 在总刚 K 中应该贡献的行索引。

$j = self.LM[e,b]$ ：同理， $self.LM[e,b]$ 给出了对应的列索引 j 。

累加贡献： $self.K[i,j] += ke[a,b]$ ：将单元刚度系数 $ke[a,b]$ 累加到总体刚度矩阵的 $[i,j]$ 位置。

关键机制：如果另一个单元也连接到相同的节点（即它的 LM 映射也指向相同的 $[i,j]$ 位置），那么在执行到那个单元时， $ke[a,b]$ 会再次被累加到 $self.K[i,j]$ 上。这样就自动实现了不同单元在共享节点处刚度的叠加，精确反映了结构真实的力学特性。

4. 位移边界条件的处理方法

两个算例均使用缩减法进行求解：

代码部分：

```
def solve_by_reduction(self, id_vec: np.ndarray):
    """缩减法求解"""
    # ===== 1 识别边界条件 =====
    # 通过标识数组分离自由和受约束的自由度
    free_dofs = np.where(id_vec == 0)[0] # 自由（未知）自由度编号
    fixed_dofs = np.where(id_vec == 1)[0] # 约束（已知）自由度编号
    # ===== 2. 矩阵分块（基于边界条件） =====
    # 根据自由度划分，对总体刚度矩阵进行分块
    K_FF = self.K[np.ix_(free_dofs, free_dofs)] # 自由-自由子块
    K_FE = self.K[np.ix_(free_dofs, fixed_dofs)] # 自由-约束子块

    # 从位移和载荷向量中提取对应部分
    d_E = self.d[fixed_dofs] # 已知位移（来自边界条件）
    f_F = self.f[free_dofs] # 自由自由度上的载荷
```

```

#检查缩减矩阵是否可解（应非奇异）
det_reduced = np.linalg.det(K_FF)
if abs(det_reduced) < 1e-10:
    print("警告：缩减刚度矩阵奇异！")
else:
    print("缩减刚度矩阵非奇异")

# ===== 3. 构建并求解缩减方程 =====
# 构建考虑边界条件后的方程右边项
rhs = f_F - K_FE @ d_E
print(f"\n 方程右边项: K_FF * d_F = {rhs}")

# 求解缩减方程: K_FF * d_F = rhs
d_F = la.solve(K_FF, rhs)

# ===== 4. 重构完整解 =====
# 将求得的未知位移赋值回完整位移向量
self.d[free_dofs] = d_F

# ===== 5. 计算约束反力 =====
R = np.zeros_like(self.f)
# 反力 = 总刚×位移 - 外载（仅在约束自由度上非零）
R[fixed_dofs] = self.K[fixed_dofs, :] @ self.d - self.f[fixed_dofs]

print(f"\n 求解完成:")
print(f" 未知位移: d_F = {d_F}")
print(f" 完整位移向量: {self.d}")

```



```
print(f' 约束反力: R = {R[fixed_dofs]}")
```

```
return self.d, R, K_FF
```

计算流程:

- 1.分离: 根据 `id_vec` 将自由度分为自由 (未知) 和约束 (已知) 两组
- 2.分块: 将原始总体刚度矩阵按自由度分组进行分块
- 3.重组: 构建只包含未知自由度的缩减方程
- 4.求解: 求解缩减方程得到未知位移
- 5.回代: 将未知位移与已知位移合并, 得到完整位移场
- 6.反力: 计算约束处为维持平衡所需的反力

5. 两个验证算例的输入数据，输出结果和结果校核

5.1 输入数据

```
example_data = {
    "Title": "算例1: 一维两单元杆结构",
    "nsd": 1,
    "ndof": 1,
    "nnp": 3,
    "nel": 2,
    "nen": 2,
    "X": [
        [0.0], # 节点1:  $x = 0$ 
        [1.0], # 节点2:  $x = 1$ 
        [2.0] # 节点3:  $x = 2$ 
    ],
    "IEN": [
        [1, 2], # 单元1: 1-2
        [2, 3] # 单元2: 2-3
    ],
    "E": [1.0, 1.0],
    "A": [1.0, 1.0],
    "EA_L": [100.0, 200.0], # 直接给出的EA/L值
    "displacement_bc": [
        {
            "node": 1,
            "dofs": [1],
            "values": [0.0]
        }
    ],
    "nodal_loads": [
        {
            "node": 2,
            "forces": [0.0]
        },
        {
            "node": 3,
            "forces": [10.0]
        }
    ]
}

with open("example1.json", "w") as f:
    json.dump(example_data, f, indent=2)
```

图 5.1 算例 1 输入数据

```

example_data = {
    "Title": "算例2：二维两杆桁架结构",
    "nsd": 2,
    "ndof": 2,
    "nnp": 3,
    "nel": 2,
    "nen": 2,
    "X": [
        [1.0, 0.0], # 节点1: (1, 0)
        [0.0, 0.0], # 节点2: (0, 0)
        [1.0, 1.0] # 节点3: (1, 1)
    ],
    "IEN": [
        [1, 3], # 单元1: 1-3
        [2, 3] # 单元2: 2-3
    ],
    "E": [1.0, 1.0],
    "A": [1.0, 1.0],
    "displacement_bc": [
        {
            "node": 1,
            "dofs": [1, 2], # UX, UY
            "values": [0.0, 0.0]
        },
        {
            "node": 2,
            "dofs": [1, 2], # UX, UY
            "values": [0.0, 0.0]
        }
    ],
    "nodal_loads": [
        {
            "node": 3,
            "forces": [10.0, 0.0] # FX=10, FY=0
        }
    ],
    "expected_results": {
        "node3_displacement": [38.284271, -10.000000],
        "stress_unit1": -10.000000,
        "stress_unit2": 14.142136
    }
}

with open("example2.json", "w") as f:
    json.dump(example_data, f, indent=2)

```

图 5.2 算例 2 输入数据

5.2 输出结果

5.2.1 算例 1 运行结果

对号矩阵 (单元自由度到总体自由度的映射):
单元1: [0 1]
单元2: [1 2]

开始组装总体刚度矩阵...

单元1: 长度 = 1.0000, 方向余弦(c,s) = (1.0000, 0.0000)
单元刚度矩阵 (形状 (2, 2)):
[100. -100.]
[-100. 100.]

单元2: 长度 = 1.0000, 方向余弦(c,s) = (1.0000, 0.0000)
单元刚度矩阵 (形状 (2, 2)):
[200. -200.]
[-200. 200.]

总体刚度矩阵 (形状 (3, 3)):
100.000000 -100.000000 0.000000
-100.000000 300.000000 -200.000000
0.000000 -200.000000 200.000000

图 5.3 算例 1 对号矩阵以及刚度矩阵的组装

施加边界条件前总体刚度矩阵行列式: 0.000000e+00
施加边界条件前总体刚度矩阵秩: 2/3

图 5.4 算例 1 施加边界条件前总体刚度矩阵行列式以及秩的计算

由计算结果可以得到算例 1 施加边界条件前总体刚度矩阵为奇异矩阵

节点位移结果	
节点	UX
1	0.000000
2	0.100000
3	0.150000
约束反力结果	
节点	RX
1	-10.000000

图 5.5 算例 1 节点位移，约束反力计算结果

5.2.2 算例 2 计算结果

```
对号矩阵 (单元自由度到总体自由度的映射):
单元1: [0 1 4 5]
单元2: [2 3 4 5]

开始组装总体刚度矩阵...

单元1: 长度 = 1.0000, 方向余弦(c,s) = (0.0000, 1.0000)
单元刚度矩阵 (形状 (4, 4)):
[0. 0. 0. 0.]
[0. 1. 0. -1.]
[0. 0. 0. 0.]
[0. -1. 0. 1.]

单元2: 长度 = 1.4142, 方向余弦(c,s) = (0.7071, 0.7071)
单元刚度矩阵 (形状 (4, 4)):
[0.35355339 0.35355339 -0.35355339 -0.35355339]
[0.35355339 0.35355339 -0.35355339 -0.35355339]
[-0.35355339 -0.35355339 0.35355339 0.35355339]
[-0.35355339 -0.35355339 0.35355339 0.35355339]

总体刚度矩阵 (形状 (6, 6)):
0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 1.000000 0.000000 0.000000 0.000000 -1.000000
0.000000 0.000000 0.353553 0.353553 -0.353553 -0.353553
0.000000 0.000000 0.353553 0.353553 -0.353553 -0.353553
0.000000 0.000000 -0.353553 -0.353553 0.353553 0.353553
0.000000 -1.000000 -0.353553 -0.353553 0.353553 1.353553
```

图 5.6 算例 2 对号矩阵以及总体刚度矩阵组装

```
施加边界条件前总体刚度矩阵行列式: 0.000000e+00
施加边界条件前总体刚度矩阵秩: 2/6
```

图 5.7 算例 2 施加边界条件前总体刚度矩阵行列式以及秩的计算

由计算结果可以得到算例 2 施加边界条件前总体刚度矩阵为奇异矩阵

节点位移结果				
节点	UX	UY		
1	0.000000	0.000000		
2	0.000000	0.000000		
3	38.284271	-10.000000		

约束反力结果				
节点	RX	RY		
1	0.000000	10.000000		
2	-10.000000	-10.000000		

模块4: 单元应力计算				
单元	长度	方向余弦(c,s)	应力	轴力
1	1.0000	(0.0000, 1.0000)	-10.000000	-10.000000
2	1.4142	(0.7071, 0.7071)	14.142136	14.142136

图 5.8 算例 2 节点位移，约束反力，单元应力计算结果

5.3 结果校核

经校核，运算结果与检查要求一致

6. 总体刚度矩阵性质分析

6.1 算例 1

经过观察图 5.3，5.4 可以得出：

对称性：算例 1 总体刚度矩阵满足对称性

奇异性：算例 1 施加边界条件前总体刚度矩阵为奇异矩阵

稀疏性：算例 1 总体刚度矩阵不是稀疏矩阵

对角元非负性：算例 1 对角元 $K_{11}=100$, $K_{22}=300$, $K_{33}=200$ ，均为正

6.2 算例 2

经过观察图 5.6，5.7 可以得出：

对称性：算例 2 总体刚度矩阵满足对称性

奇异性：算例 2 施加边界条件前总体刚度矩阵为奇异矩阵

稀疏性：算例 2 总体刚度矩阵是稀疏矩阵

对角元非负性：算例 2 对角元也均为正

7. 总体刚度矩阵第 j 列物理意义说明

总体刚度矩阵 K 的第 j 列，表示当第 j 个自由度产生单位位移（其他自由度位移均为 0）时，在所有自由度上产生的节点力。